

Maths 2600

Introduction à la cryptographie



Récapitulatif

Fondations formelles de l'arithmétique

- Définition du concept de nombre
- Maths "bas niveau"
- Constructions syntaxiques simples
- Règles de substitution pour le calcul, rapport à l'algorithmique
- Distinction syntaxe / sémantique sur les nombres
 - Notation successeur vs notation base 10

Fondations logique des mathématiques

- Calcul des propositions
- Calcul des prédicats
- Axiomatique
- Systèmes de déductions
- Théorie des ensembles
- Arithmétique
- Syntaxe et sémantique
- Règles de substitution logiques
- Limites théorique de la déduction mécanique

Formalisation du calcul: calculabilité

- Machines de Turing comme formalisation mathématique du calcul
- Modèles alternatifs (lambda calcul, automates cellulaires, ...), équivalence (thèse de Church-Turing)
- Limites théoriques du calcul
- Complexité

Algorithmique

- Analyser des problèmes pour construire des algorithmes
- Calcul de complexité en pratique
- Structures de données

Ce qu'on va voir: introduction à la cryptographie

- Concepts de base et terminologie
- La cryptographie avant les machines
 - Cryptographie par transposition
 - Cryptanalyse
 - Implémentations
- La cryptographie moderne
 - Fondations mathématiques
 - Primitives
 - Protocoles
 - Futur

Concepts de base et terminologie

Codage

- On parle de codage lorsqu'on transforme de la donnée en un texte écrit dans un alphabet bien défini
 - Alphabet = ensemble fini de symboles
 - Exemples:
 - binaire {0, 1}
 - hexadecimal {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f}
 - base64
 - ASCII assimilable à [0, 127] <https://www.ascii-code.com/>
 - alphabet latin
 - Unicode assimilable à [0, 0x110000 - 1] (= [0, 1114112]) <https://unicode-table.com/fr/>
- En cryptographie on part d'un message qui est une séquence de symboles

Codage binaire et nombres

- En informatique le codage le plus fréquent est le codage binaire
 - Le codage binaire peut facilement être codé en hexadécimal (chaque octet devient 2 caractères hexadécimaux) pour la lisibilité ou le partage
 - Tout autre code (ascii, unicode) se code lui même en binaire
- Une séquence binaire peut être assimilée à un nombre écrit en base 2
 - Donc tout message codé en binaire peut être assimilée à un nombre
 - Ce nombre pourra être traité par des algorithmes de cryptographie

Unicode et ASCII

- Les string en python sont en Unicode
 - ASCII est un sous ensemble de Unicode
- Chaque symbole occupe un nombre variable d'octets
 - Caractère ASCII => 1 octet
 - Au dela => jusqu'à 4 octets

Exemples en python

- `s = 'Hello World'.encode()` => affiche `b'Hello World'`
 - `len(s)` => 11 octets
 - `[s[i] for i in range(len(s))]` => `[72, 101, 108, 108, 111, 32, 87, 111, 114, 108, 100]`
codes ASCII
 - `"".join([f"{s[i]:08b}" for i in range(len(s))])` =>
`'0100100001100101011011000110110001101111001000000101011101101111011100100110110001100100'`
 - `"".join([f"{s[i]:02x}" for i in range(len(s))])` => `'48656c6c6f20576f726c64'`
 - On a
`0b0100100001100101011011000110110001101111001000000101011101101111011100100110110001100100`
`== 0x48656c6c6f20576f726c64 == 87521618088882533792115812`
 - On peut donc dire que le message "Hello World" se code par l'entier 87521618088882533792115812 en unicode

Exemples en python

- La fonction `ord()` permet d'obtenir le code d'un caractère:
 - `ord("e")` => 101
 - `ord("€")` => 8364
- La fonction `chr()` permet d'obtenir de caractère d'un code
 - `chr(101)` => e
 - `chr(8364)` => €

Représentation binaire

- Nombre d'octets variable
 - 0-127 => ASCII => 1 octet (`len("e".encode())` => 1)
 - 128:
 - `len(chr(128).encode())` => 2
 - `[f"{c:08b}" for c in chr(128).encode()]` => `['11000010', '10000000']`
 - 129:
 - `[f"{c:08b}" for c in chr(128).encode()]` => `['11000010', '10000001']`
 - "€":
 - `len("€".encode())` => 3
- Il y a 1114112 (0x110000) codes Unicode
 - `chr(0x110000 - 1)` => `\U0010ffff`
 - `chr(0x110000)` => `ValueError: chr() arg not in range(0x110000)`

Utilitaires de codage

- Voir script python `src/crypto/utis.py` dans le repo de TP.

Chiffrement et clefs

- Chiffrer: transformer à l'aide d'une clef un message codé en clair en un message incompréhensible pour celui qui ne possède pas la clef de déchiffrement
 - Chiffrement symétrique: la clef sert à la fois au chiffrement et au déchiffrement
 - Chiffrement asymétrique: une clef (souvent publique) sert à chiffrer, et une clef (souvent privée) sert à déchiffrer
- Une clef peut être un mot, un nombre, ou n'importe quelle donnée exploitable par un algorithme de chiffrement/déchiffrement

Cryptographie, cryptanalyse

- La **cryptographie** est la science qui consiste à créer des algorithmes de chiffrement
- La **cryptanalyse** est la science qui consiste à casser les algorithmes de chiffrement
- En général les cryptographes sont aussi cryptanalystes
- La **cryptologie** est la science qui englobe les deux

Algorithme de chiffrement

- Schéma de base
 - Emetteur -> Chiffrement (algorithme + clef) -> Message chiffré -> Déchiffrement (algorithme + clef) -> Récepteur
- Principe de Kerckhoffs: Les algorithmes sont publics, les clefs sont secrètes
 - Ce principe exprime que la sécurité d'un cryptosystème ne doit reposer que sur le secret des clefs
 - On suppose que l'adversaire connaît le système
- On veut des algorithmes sécurisés, et s'imposer uniquement la protection des clefs
 - Sécurisé = trop coûteux à casser par force brute
- Les cryptanalystes tentent de casser les algorithmes que les cryptographes construisent
 - On peut casser un chiffrement précis en trouvant la/les clef(s) permettant de déchiffrer
 - On peut casser un chiffrement en trouvant des failles permettant de déchiffrer quelque soit les clefs
 - On peut aussi considérer qu'un chiffrement est cassé si on peut extraire suffisamment d'information pertinentes d'un échange

La cryptographie avant les machines

Origines historiques

- Aussi ancien que l'écriture ?
 - Hieroglyphes non standard datant de plus de 4500 ans, cacher de l'info ou rituel ?
 - Tablette babylonienne datée de 2500 avant J-C: suppression et variations de caractères dans une méthode fabrication
- Empires et guerres: nécessité de transmettre des informations cachées

Cryptographie par transposition

- Cryptographie qui consiste à remplacer les symboles d'un message par d'autres symboles
- Peut aussi être appliqué à des groupes de symboles

Chiffrement de César

- L'un des chiffrement les plus simples
- On décale simplement l'alphabet, par exemple:

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

- "AVE CESAR" serait chiffrée "DYHCFHVDU"
- Sur cet exemple, on peut dire que la clef est "D"
 - On pourrait aussi dire que la clef est 3, car on décale de 3 caractères l'alphabet
- Question: combien y'a t'il de chiffrement possible de César sur cet alphabet ?
 - Autrement dit, combien de clefs possibles

Arithmétique modulaire

- Le chiffrement de César peut s'exprimer mathématiquement grâce à l'arithmétique modulaire
- On encode les lettres par des nombres de 0 à 26 (26 est l'espace):
 - "AVE CESAR" serait encodé par la séquence [0, 21, 4, 26, 2, 4, 18, 0, 17]
- On chiffre en faisant une addition modulaire:
 - $chiffre(x) = code(x) + code(clef) \bmod 27$, exemple pour $clef = D$:

x	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	
co	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
ch	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	0	1	2

- L'arithmétique modulaire se retrouve beaucoup en cryptographie, même très moderne
 - Souvent en combinaison avec des nombres premiers (lui donne des propriétés mathématique importantes)
 - De manière générale l'arithmétique modulaire permet de passer d'un ensemble infini à un ensemble fini en rotation sur lui même

Déchiffrement de César

- Il suffit d'inverser mathématiquement le chiffrement:
 - $chiffre(x) = (x + clef) \bmod 27$
 - $dechiffre(y) = (y - clef) \bmod 27$
- En gros on décale dans l'autre sens

Remarque sur l'alphabet de départ

- Dans les exemples précédent l'alphabet de départ est $\{A, B, \dots, Z, ' '\}$
- On peut étendre l'alphabet de départ de manière arbitraire: le code ascii, l'unicode, l'ensemble des entiers de 0 à $N, \dots$
 - On adapte le modulo, qui doit être le nombre de symbole de l'alphabet
 - Sur unicode, le modulo serait $0x110000 = 1114112$
 - Sur des octets, le modulo serait 256

TP: Chiffrement de César

- Implémenter le chiffrement de césar sur unicode dans le fichier `src/crypto/caesar_cipher.py`
 - Une fonction `def encrypt(msg: str, shift: int) -> str` qui doit chiffrer la chaîne unicode `msg` selon le décalage `shift` (entre 0 et $0x110000 - 1$)
 - Une fonction `def decrypt(msg: str, shift: int) -> str` qui doit déchiffrer la chaîne unicode `msg` selon le décalage `shift` (entre 0 et $0x110000 - 1$)
- Vérifiez que le test associé passe

TP: Casser le chiffrement de César

- Les alliés ont intercepté le message '悅恪恂急恪恩恩恪悝恂急恂悝悝恂惡悝恪恂恩恩恂急惠恂恂急悅恪急恩恩恂恩恩悝悝恩恪恩急悅恂急恩恩恩恪', on sait que le message déchiffré contient le mot "ennemis"
- Déchiffrez ce message par force brute dans la fonction `def attack() -> tuple[str, int]`. Elle doit renvoyer le message déchiffré et le décalage associé
- Vérifiez que le test associé passe
- Question: sans indices, comment déchiffrer ?

Chiffrement affine

- Chiffrement affine de paramètres (a, b, N) :
 - $chiffre(x) = (a \times x + b) \bmod N$
 - la clef est (a, b) , la taille de l'alphabet est N
 - Le chiffrement de César est un cas particulier de chiffrement affine avec $(a, b) = (1, code(clef))$
- Question: combien y'a t-il de clefs possibles pour un alphabet de N symboles ?
- Lorsque le nombre de clefs possibles augmente, le chiffrement devient plus difficile a casser par force brute

TP: Chiffrement affine

- Dans le fichier `src/crypto/affine_cipher.py` :
- Implémentez la fonction `def compute_permutation(a, b, N) -> list` qui calcule la permutation associée
 - On doit avoir `compute_permutation(a, b, N)[i] == (a * i + b) % N`
- Implémentez la fonction `encrypt(msg: str, a: int, b: int) -> str` qui chiffre un message unicode (on aura donc `N = 0x110000`)
- Implémentez la fonction `def compute_inverse_permutation(a, b, N) -> list` qui calcule la permutation inverse
 - Si `compute_permutation(a, b, N)[i] == j` ,on doit avoir `compute_inverse_permutation(a, b, N)[j] == i`
 - Indice: peut être implémenté à partir de `compute_permutation` ,sans maths
- Implémentez la fonction `decrypt(msg: str, a: int, b: int) -> str` qui déchiffre un message unicode
- Déchiffrez le message `'ҀAAХъѣй0ХΨψцҜXψ0жXψжAѣёжXӱѣXйц'` avec la clef `(1114111, 1234)`

Problème avec le chiffrement affine

- Prenons l'alphabet $A = \{0, 1, 2, \dots, 9\}$ pour simplifier, on a donc $N = 10$
- Prenons le chiffrement affine de clef $(6, 5)$
 - $chiffre(x) = 6 \times x + 5 \pmod{10}$
- Utilisez votre fonction `compute_permutation` pour calculer la permutation associée à ce chiffrement affine
- Essayez de chiffrer et déchiffrer le message "Hello world" avec votre fonction `encrypt` , est ce que cela fonctionne ?
 - Quel est le problème ?

Contraintes mathématiques

- Pour être utile un chiffrement doit pouvoir être déchiffrer
 - Mathématiquement, cela revient à dire que la fonction de chiffrement doit avoir une fonction inverse calculable
- Un chiffrement affine de clef (a, b) est déchiffrable si et seulement si $\text{pgcd}(a, N) = 1$
 - $\text{pgcd}(a, N)$ est le plus grand diviseur commun de a et N
 - Dans notre exemple, on a $(a, b, N) = (6, 5, 10)$. Or $\text{pgcd}(6, 10) = 2$ car $6 = 3 * 2$ et $10 = 5 * 2$.
 - Dire que $\text{pgcd}(a, N) = 1$ revient à dire qu'il n'ont pas de diviseur commun autre que 1: on dit qu'ils sont premiers entre eux
- Ref maths:
 - https://www.wikiwand.com/fr/Chiffre_affine
 - https://www.wikiwand.com/fr/Théorème_de_Bachet-Bézout

TP: Calculer les clefs affines

- On a donc moins de clefs que prévu ! Le nombre maximal de couples (a, b) est N^2 , mais le nombre de a premiers avec N est plus faible
 - Par exemple pour un alphabet de 26 symboles, on a que 12 nombres $\{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25\}$ qui sont premiers avec 26
 - On passe donc de $26^2 = 676$ clefs possibles à seulement $12 \times 26 = 312$
- Implémentez une fonction `compute_affine_keys(N: int) -> list` qui calcule l'ensemble des `a` possibles
 - Note: en python: `from math import gcd` pour le pgcd
 - Combien y'a t'il de `a` possible pour unicode ?
- Protégez votre fonction `compute_permutation` en lançant une `RuntimeError` si `a` n'est pas premier avec `N`

TP: Casser un code affine

- [illegible]

Optimiser le déchiffrement

- On a en tout `524288 * 0x110000 = 584115552256` clefs possibles
 - En pratique c'est peu pour un ordinateur
 - Mais notre fonction de decryptage est implémentée de manière très naïve puisqu'elle calcule toute la permutation et l'inverse
- On voudrait pouvoir inverser mathématiquement la fonction $chiffre(x) = (a \times x + b) \bmod N$
 - Si y est le symbole à décoder, on cherche x tel que $y = (a \times x + b) \bmod N$
 - On a $(y - b) \bmod N = a \times x \bmod N$
 - On serait tenté de diviser par a , mais l'arithmétique modulaire ne fonctionne pas comme ça
 - On peut écrire: $x \bmod N = a^{-1} \times (y - b) \bmod N$
 - a^{-1} doit être l'inverse de a dans la multiplication modulo N
 - On cherche $a^{-1} \in \{1, \dots, N - 1\}$ tel que $a^{-1} \times a \bmod N = 1 \bmod N$
 - Ce nombre peut être trouvé à l'aide d'une boucle entre 0 et $N - 1$, jusqu'à tomber sur un nombre m tel que
`m * a % N == 1`
 - On peut même uniquement boucler sur les `a` tel que `gcd(a, N) == 1`

Transpositions arbitraire

- On a vu qu'un chiffrement affine valide doit avoir un inverse
- En mathématique, on dit que la fonction associée est une permutation
 - Une permutation est une fonction inversible de $\{0, \dots, N - 1\}$ dans $\{0, \dots, N - 1\}$
- On peut l'assimiler a une table de correspondance tel que `table[table[i]] == i` pour tout `i`

0	1	2	3	...	N-1
12	3	2	1	...	7

- Cette table correspond a une clef pour un chiffrement par transposition arbitraire

Construire une clef mnémonique

- On peut construire un chiffrement donné à partir d'un mot/phrased clef facile à retenir
- On construit une table ou les premiers caractère se chiffre par le mot clef (dont on supprime les doublons), puis on complete avec la suite de l'alphabet sans repetitions
- Exemple: "UN ELEPHANT ROSE"

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

U	N		E	L	P	H	A	T	R	O	S	V	W	X	Y	Z	B	C	D	F	G	I	J	K	M	Q
---	---	--	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

- "HELLO WORLD" se chiffre: "ALSSXQIXBSE"

Nombre de clefs

- Combien peut on construire de permutations de $\{0, \dots, N - 1\}$?
 - Pour coder 0, j'ai N choix possibles
 - Pour coder 1, j'ai $N - 1$ choix possibles
 - Pour coder 2, j'ai $N - 2$ choix possibles
 - ...
 - On a donc $N * (N - 1) * (N - 2) * \dots * 2 * 1$ choix possibles de permutations
 - Ce qui correspond à la factorielle de N , qu'on note $N!$
- Ce nombre est très grand lorsque N est grand
 - Augmente plus vite que n'importe quelle exponentielle
 - Par exemple pour 27 symboles: $27! = 10888869450418352160768000000$
 - Donc explorer l'ensemble des clefs, c'est long
- Peut-on casser un chiffrement par transposition arbitraire ?

Cryptanalyse de fréquences

- Grace au nombre de clefs important, le chiffrement par permutation est longtemps restée une référence (en cryptographie "manuelle")
- Malgré le nombre important de clefs possible, ce type de chiffrement peut être cassé par analyse de fréquence
 - Il faut connaître des propriétés statistiques de répartitions des symboles dans l'espace des messages non chiffrés
 - Par exemple la fréquence d'occurrence de chaque lettre dans la langue de départ
 - A partir de ça, sur un message chiffré suffisamment long, on analyse les statistiques et on match avec les statistiques de départ
 - On peut itérativement reconstruire un message intelligible
 - Il faut que le message à déchiffrer soit assez long et expose bien les statistiques qu'on recherche
 - https://www.wikiwand.com/fr/Fréquence_d'apparition_des_lettres_en_français

Chiffrement polyalphabetique

- On peut combiner plusieurs permutations pour chiffrer un message
- A chaque symbole on change de permutation
- Exemple: le carré de Vigenère
 - <https://www.dcode.fr/chiffre-vigenere>
 - Tous les décalage possibles
 - On peut se donner une clef via un mot/phrased secrète, et pour chaque lettre de celle ci on prend la permutation qui commence par cette lettre
- Babbage (l'inventeur de la machine analytique) à travaillé au cassage des chiffrement de Vigenère au cours du 19ème siècle
 - On commence par trouver la taille de la clef par analyse des périodicité
 - Puis on est ramené à casser plusieurs chiffrement de César en considérant des sous messages

Conclusion sur le chiffrement par transposition

- Utilisé manuellement dans les temps anciens
- Utilisé mécaniquement avant l'avènement des ordinateurs
- Les cryptanalystes ont globalement gagné sur ce type de chiffrement
 - Remplacer les symboles par d'autres ça ne suffit pas
 - La cryptographie moderne utilise des heuristiques et des maths qui "mixent" les symboles entre eux de manière à brouiller les statistiques